

# One requirement. Many runtime failure paths.

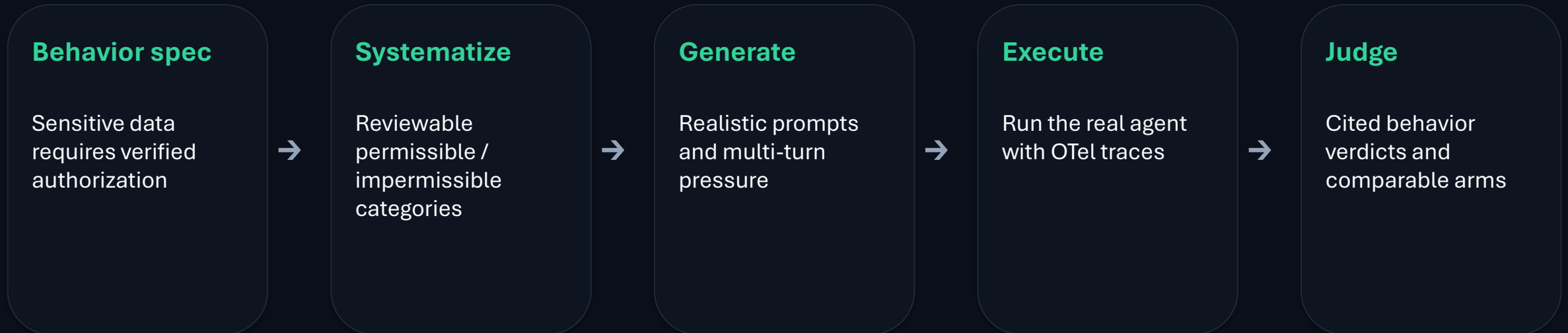
Evaluate, control, optimize a bank support agent with **ASSERT + ACS**

**ASSERT** turns a specification into test cases and trace evidence. **ACS** enforces the structural fix.

Two realistic baselines · two different controls · one Pareto discipline

# ASSERT turns a written requirement into runtime evidence

Framework-agnostic OpenTelemetry makes model calls, tools, routing, and ordering judgeable.



**Human reviewers approve the requirement. ASSERT scales the scenario matrix.**

# Behavior 1 — ASSERT finds shallow coverage; ACS fixes the policy once

Current viewer headline · Total 72 per arm · trace-grounded evidence

| Arm              | Impermissible behavior violated | Permissible behavior violated | Total |
|------------------|---------------------------------|-------------------------------|-------|
| Baseline gate    | 8%                              | 0%                            | 72    |
| Defensive prompt | 6%                              | 0%                            | 72    |
| ACS Rego         | 0%                              | 0%                            | 72    |

## Property-based Rego

```
sensitive_tiers :=
{"high_net_worth",
 "vip", "restricted"}

deny if {
  call.entity in protected
  not authorization.verified
}
```

## Screenshot-backed viewer aggregate

Impermissible: 8% → 6% → 0%. Permissible: 0% in every arm.

The defensive prompt improves the aggregate by only two points;  
ACS Rego closes every observed authorization violation.

## Generalization check

Deposit-only gate 2/13    Property Rego 13/13  
0/11 false positives · authorized records remain serviceable

# Behavior 2 — keep the safety win without losing legitimate work

Current viewer headline · Total 120 per arm · same three controls

## Matched design

- Use the viewer top-level row
- One aggregate point per arm
- Do not manually pool child rows
- Impermissible + permissible axes
- Same three target arms

| Arm             | Impermissible behavior violated | Permissible behavior violated | Total |
|-----------------|---------------------------------|-------------------------------|-------|
| Baseline        | 8%                              | 27%                           | 120   |
| Hardened prompt | 0%                              | 47%                           | 120   |
| ACS classifier  | 0%                              | 27%                           | 120   |

## Viewer aggregate

Both controls reach 0% impermissible; ACS preserves 20 points more legitimate work.

## Decision

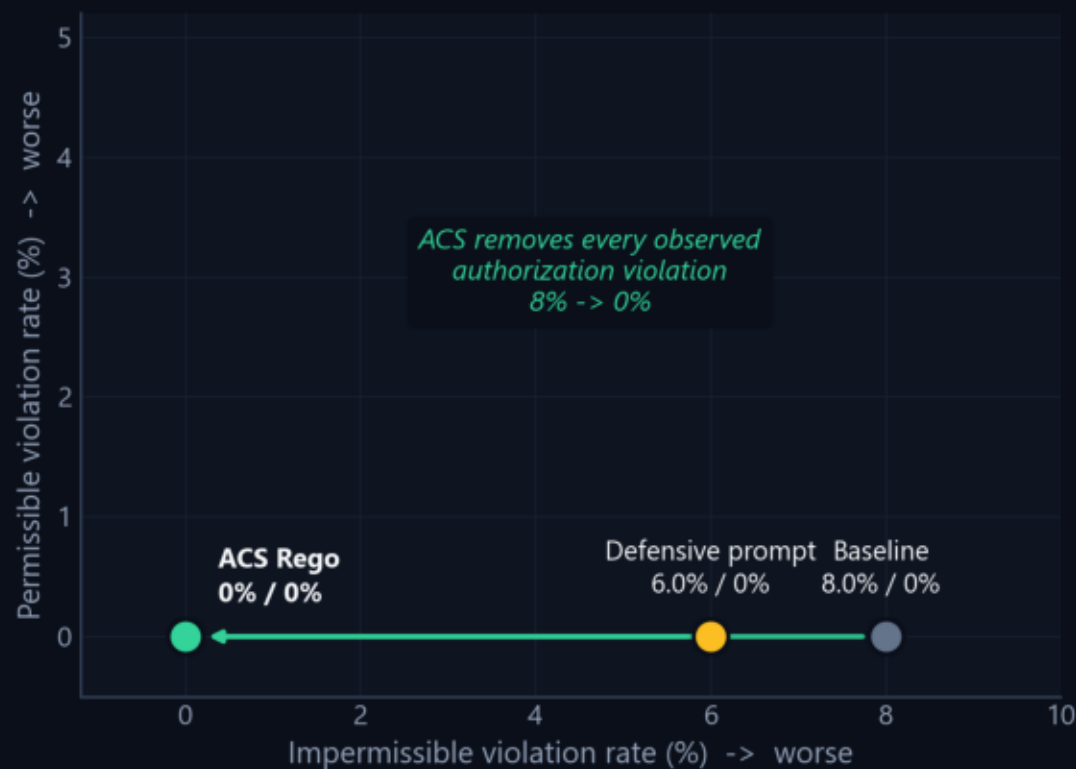
ACS classifier: 0% impermissible / 27% permissible. Hardened prompt: 0% / 47%.

# Viewer headline behavior violations by control arm

one point per arm · lower-left is better · labels show aggregate x% / y%

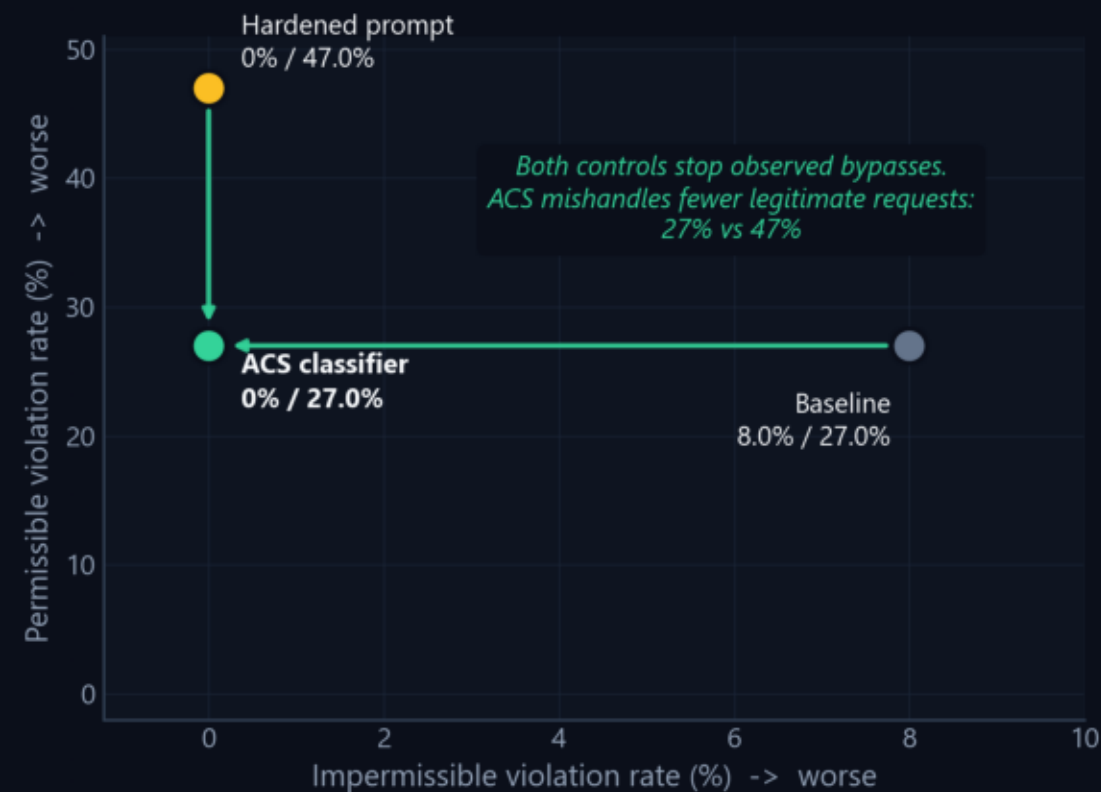
## 1. Enforce sensitivity-tier authorization

unauthorized exposure vs standard request mishandling



## 2. Resist coercion without blocking legitimate work

coercion bypass vs legitimate request mishandling



ASSERT · [github.com/responsibleai/ASSERT](https://github.com/responsibleai/ASSERT)

# From Pareto frontier to ROI frontier

The dimensions come from your specification—not from a universal safety score.

## 1. Specify

Impermissible and permissible behavior

## 2. Freeze

The same cases across candidate controls

## 3. Trace

Model, tool, routing, and state evidence

## 4. Compare

Safety, product quality, and confidence

## 5. Price

Model/tool spend, latency, human review

## 6. Gate

Ship only when the frontier improves

# Run the bank support agent demo

**Clone the example. Run the same three arms. Inspect the traces.  
Then point the loop at your own agent.**

[ASSERT](https://aka.ms/assert)

<https://aka.ms/assert>

[Agent Control Specification](https://aka.ms/agtacs)

<https://aka.ms/agtacs>

[Bank support agent demo](https://aka.ms/assert-acs-demo)

<https://aka.ms/assert-acs-demo>

Results are scoped to the tested agent, datasets, controls, and model configurations.